ESPECIFICAÇÃO DE ARQUITETURA DO SISTEMA

TaskHub — Planner de Equipe Colaborativo

Documento Técnico de Arquitetura

Sumário

Sumário 2
1. Informações do Documento 3
1.1 Dados Gerais 4
1.2 Controle de Versões 4
1.3 Referências 4
1.4 Glossário e Siglas 4
2. Sumário Executivo 6
2.1 Visão Geral 6
2.2 Objetivos Estratégicos 6
3. Introdução 6
3.1 Contexto 6
3.2 Escopo 6
3.3 Premissas 6
3.4 Restrições 7
3.5 Stakeholders 7
4. Arquitetura da Solução 8
4.1 Visão Geral da Arquitetura 8
Objetivos Arquiteturais 8
Diagrama de Arquitetura Geral 8
Descrição das Camadas 8
4.2 Componentes da Arquitetura 8
5. Modelo C4 11
5.1 Contexto (C1) 11
Objetivo 11
Elementos 11
Diagrama C1 11
5.2 Contêineres (C2) 11
Objetivo 11
Elementos 12
Diagrama C2 12
5.3 Componentes (C3) 12
Objetivo 12
Componentes 12
Diagrama C3 13
6. Arquitetura de Dados 14
6.1 Fontes de Dados 14
6.2 Modelo de Dados 14
Diagrama de Dados 14

7. Integrações 16
7.1 Integrações Internas 16
7.2 Integrações Externas 16
7.3 Mensageria 16
8. API 17
8.1 Padrões 17
8.2 Catálogo de APIs 17
8.3 Segurança das APIs 18
8.4 OpenAPI / Swagger 18
9. Arquitetura de IA e MLOps (Se aplicável) 19
9.1 Modelos Analíticos 19
9.2 Fluxo de Treinamento 19
9.3 Fluxo de Inferência 19
9.4 Governança de Modelos 19
9.5 Monitoramento 19
10. Segurança 20
10.1 Princípios 20
10.2 Autenticação 20
10.3 Autorização (RBAC/ABAC) 20
10.4 Gestão de Segredos 20
10.5 Proteção de Dados 21
10.6 Auditoria 21
10.7 Compliance 21
11. Observabilidade 22
11.1 Monitoramento 22
11.2 Logging 22
11.3 Tracing 22
11.4 Alertas 22
11.5 Dashboards Operacionais 22
12. ADRs – Architecture Decision Records 23
ADR-001 23
Decisão 23
Contexto 23
Alternativas Avaliadas 23
Justificativa 23
Consequências 23

1. Informações do Documento

1.1 Dados Gerais

Campo	Valor
Projeto	TaskHub — Planner de Equipe (pacote team-planner)
Cliente	A definir
Contrato	A definir
Versão	1.0 (aplicação v1.0.0)
Data	30/10/2026
Elaborado por	Kaike Henrique, Gabriel Estevo, Gregory Yamamoto e João Rossi
Revisado por	A definir
Aprovado por	A definir

1.2 Controle de Versões

Versão	Data	Autor	Descrição
1.0	30/10/2026	Kaike Henrique, Gabriel Estevo, Gregory Yamamoto e João Rossi	Versão inicial, elaborada a partir da análise do código-fonte (front-end e schema Prisma)

1.3 Referências

- README.md do projeto — visão funcional, stack, perfis de acesso e estrutura de pastas
- prisma/schema.prisma — modelo relacional proposto (PostgreSQL)
- package.json e package-lock.json — dependências e versões efetivamente resolvidas
- Código-fonte em src/ (types.ts, store/useAppStore.ts, components/, lib/utils.ts, mock/mockData.ts)
- Documentação oficial de React 18, Vite 5, Zustand 4, Prisma 5, Tailwind CSS 3 e date-fns 3
- Lei nº 13.709/2018 (LGPD) e modelo C4 de documentação de arquitetura

1.4 Glossário e Siglas

Termo/Sigla	Descrição
SPA	Single Page Application — aplicação web carregada uma única vez no navegador, com navegação feita no cliente.
RBAC	Role-Based Access Control — controle de acesso por papel. No sistema, os papéis são LEADER (Líder/Gestor) e MEMBER (Colaborador).
Kanban	Quadro visual com as colunas A Fazer, Em Andamento, Revisão e Concluído, no qual o status da tarefa muda por arrastar e soltar.
Store	Estado global da aplicação (useAppStore), implementado com Zustand. É a fonte única de verdade e concentra as regras de negócio.

Termo/Sigla	Descrição
ORM	Object-Relational Mapping — mapeamento objeto-relacional. O projeto propõe o Prisma ORM.
Tarefa privada	Tarefa pessoal (isPrivate = true), visível apenas para o usuário que a criou.
Subtarefa / Checklist	Itens que compõem uma tarefa. O percentual concluído define o progresso da tarefa.
Activity Log	Histórico de ações realizadas em uma tarefa (criação, mudança de status, prazo, prioridade, atribuição, comentário e checklist).
JWT	JSON Web Token — token assinado proposto para autenticação da API.
C4	Modelo de documentação de arquitetura em níveis: Contexto, Contêineres, Componentes e Código.
ADR	Architecture Decision Record — registro de decisão arquitetural.
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018).

2. Sumário Executivo

2.1 Visão Geral

- **Contexto do projeto:** o TaskHub é uma aplicação web de planejamento e gestão de tarefas para equipes, com calendário (mês, semana e dia), quadro Kanban, checklist de subtarefas, comentários, histórico de atividades e um painel de carga de trabalho para líderes. A versão atual (1.0.0) é um front-end funcional em React + TypeScript, que usa dados simulados salvos no navegador. O modelo de banco de dados já está definido em Prisma para a futura conexão com um back-end real.
- **Problema que será resolvido:** falta de visibilidade centralizada de prazos, responsáveis e andamento das entregas da equipe, além da dispersão de comunicação e histórico sobre cada tarefa.
- **Benefícios esperados:** organização das entregas em uma única ferramenta; identificação imediata de tarefas atrasadas; acompanhamento da carga de trabalho por colaborador; rastreabilidade das alterações; e separação entre tarefas de equipe e tarefas pessoais.

2.2 Objetivos Estratégicos

- **Objetivos de negócio:** aumentar a previsibilidade das entregas, reduzir atrasos, apoiar a gestão de capacidade da equipe pelo líder e registrar a comunicação no contexto de cada tarefa.
- **Objetivos tecnológicos:** evoluir o protótipo front-end para uma solução completa com API, banco PostgreSQL, autenticação corporativa e atualização em tempo real entre usuários, sem precisar reescrever os componentes de interface.
- **Metas de arquitetura:** manter a store como fronteira única entre a interface e os dados; validar entradas com Zod na borda da API; aplicar RBAC no servidor; e garantir trilha de auditoria por meio do Activity Log.

3. Introdução

3.1 Contexto

Equipes que trabalham em vários projetos ao mesmo tempo precisam acompanhar prazos, prioridades e responsáveis em um só lugar. O TaskHub atende essa necessidade com um planner colaborativo com dois perfis: o Líder/Gestor, que cria e atribui tarefas de equipe e acompanha a carga de trabalho, e o Colaborador, que executa tarefas, atualiza o status, comenta e organiza suas tarefas pessoais. A versão atual é um protótipo funcional de front-end. A persistência ocorre no localStorage do navegador e a troca de usuário é simulada para demonstrar os dois perfis.

3.2 Escopo

- **Implementado (v1.0.0):** Planner com visões Mês, Semana e Dia e reagendamento por arrastar e soltar; Kanban com 4 colunas; criação de tarefas com título, descrição, prazo com hora, prioridade, etiquetas, projeto e múltiplos responsáveis; detalhe da tarefa com abas Detalhes, Checklist, Comentários e Histórico; tarefas privadas; filtros por colaborador e por projeto; Painel da Equipe exclusivo do líder; persistência local.
- **Proposto (evolução):** API back-end, banco PostgreSQL via Prisma, autenticação real, validação com Zod, atualização em tempo real (WebSockets/SSE) e observabilidade.
- **Fora do escopo:** aplicativo mobile nativo, notificações por e-mail/push, anexos de arquivos, relatórios exportáveis e recursos de IA.

3.3 Premissas

- Os usuários acessam por navegador moderno, em resolução de desktop (o layout é otimizado para telas a partir de 1024 px).
- Cada usuário pertence a uma única equipe (Team) e tem um único papel: LEADER ou MEMBER.
- Os componentes de interface conhecem os dados apenas pela store; por isso, as ações da store podem ser trocadas por chamadas de API sem alterar a interface.
- O banco de produção será PostgreSQL. O SQLite é aceito apenas para protótipo local, com as adaptações descritas no schema.
- A interface e as datas são exibidas em português do Brasil (pt-BR).

3.4 Restrições

- Não existe back-end nem autenticação real. A sessão é simulada pelo seletor de usuário no Topbar e as regras de permissão são aplicadas apenas no cliente.
- Os dados ficam no localStorage (chave "team-planner-storage"), limitados ao navegador e ao dispositivo de cada usuário, sem compartilhamento entre pessoas.
- Os pacotes @prisma/client e zod estão instalados, mas ainda não são utilizados pelo código.
- Não há colaboração em tempo real: a "atualização ao vivo" acontece apenas dentro da mesma sessão do navegador.
- Build e execução exigem Node.js 18 ou superior (requisito do Vite 5).

3.5 Stakeholders

Nome	Papel	Área/Empresa	Contato
Kaike Henrique	Integrante da equipe do projeto	Equipe TaskHub	A definir
Gabriel Estevo	Integrante da equipe do projeto	Equipe TaskHub	A definir
Gregory Yamamoto	Integrante da equipe do projeto	Equipe TaskHub	A definir
João Rossi	Integrante da equipe do projeto	Equipe TaskHub	A definir
A definir	Líder / Gestor (usuário-chave)	A definir	A definir
A definir	Colaboradores (usuários finais)	A definir	A definir

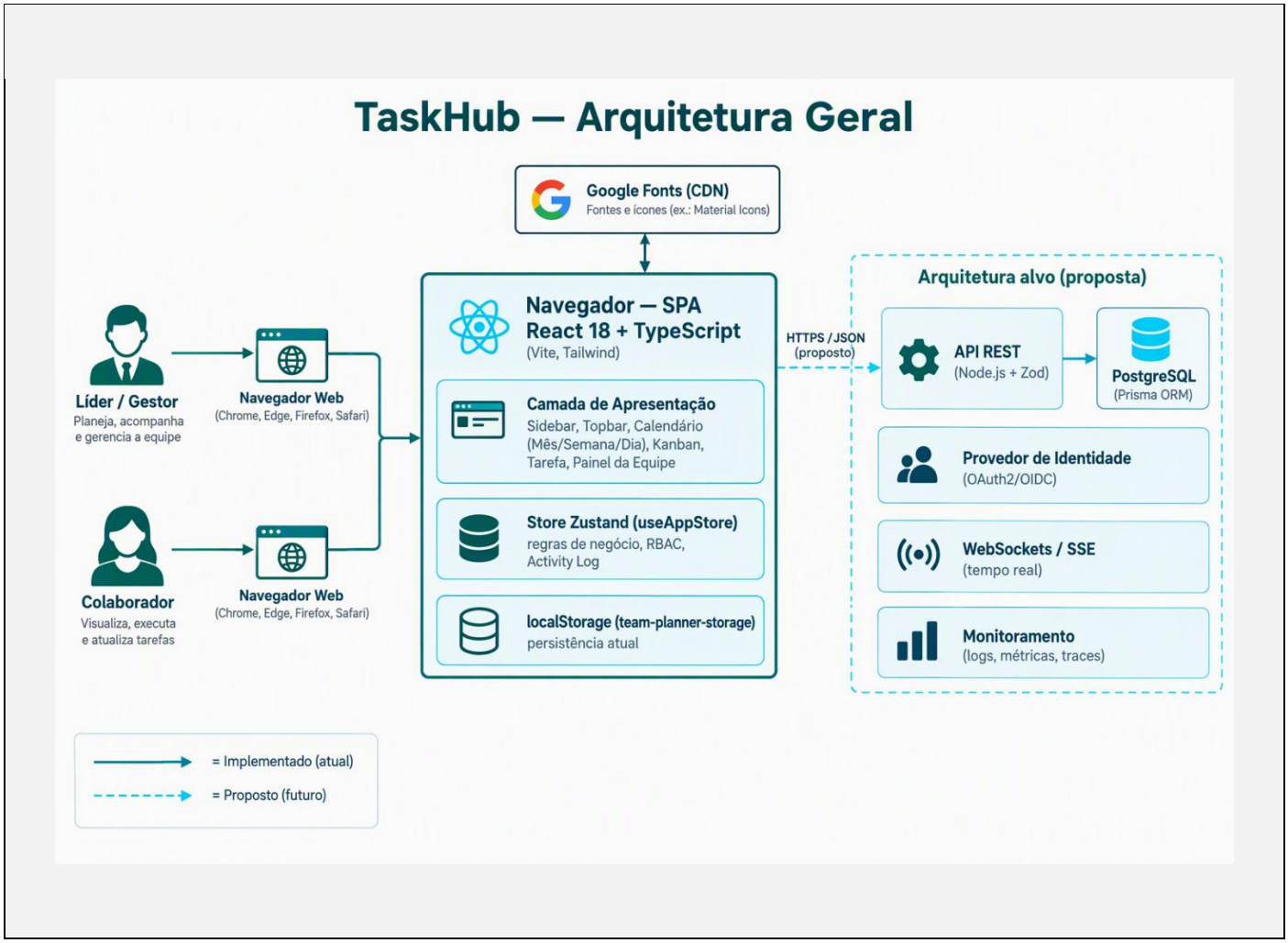
4. Arquitetura da Solução

4.1 Visão Geral da Arquitetura

Objetivos Arquiteturais

- **Disponibilidade:** hoje a aplicação é estática e funciona sem servidor de dados. Na arquitetura alvo, o front-end fica em hospedagem estática com CDN e a API é stateless, permitindo múltiplas instâncias.
- **Escalabilidade:** API sem estado com escala horizontal; banco PostgreSQL com índices por equipe, projeto, prazo e status (já definidos no schema).
- **Segurança:** RBAC por papel (LEADER/MEMBER), tarefas privadas visíveis só ao criador e autenticação por token. As regras deverão ser aplicadas no servidor.
- **Performance:** renderização reativa com seletores do Zustand e useMemo; filtros e agrupamentos feitos em memória; bundle otimizado pelo Vite com Tailwind gerando apenas as classes usadas.
- **Governança:** tipos de domínio centralizados em types.ts, espelhando o schema Prisma; histórico de atividades por tarefa; dependências com versão travada no package-lock.json.

Diagrama de Arquitetura Geral



Descrição das Camadas

- **Apresentação:** SPA em React 18 + TypeScript, estilizada com Tailwind CSS (paleta própria: ink, paper, pine, amber, clay) e ícones lucide-react. É composta por Layout (Sidebar e Topbar), Calendário (CalendarView, MonthView, WeekView e DayView), Kanban, módulo de Tarefa e Painel da Equipe.

- **Serviços:** hoje, a store Zustand (useAppStore) funciona como camada de serviço no cliente, com sessão, visibilidade, CRUD de tarefas, subtarefas, comentários e log de atividades. Na proposta, essas ações passam a chamar uma API REST.
- **Integração:** hoje, apenas a Web Storage API (localStorage) e o Google Fonts. Na proposta, HTTPS/JSON entre front-end e API e WebSockets/SSE para eventos em tempo real.
- **Dados:** hoje, o estado em memória é salvo como JSON no localStorage, com carga inicial do mockData. Na proposta, PostgreSQL acessado via Prisma ORM, com 8 entidades.
- **Analytics:** indicadores do Painel da Equipe calculados em tempo de execução (total, em andamento, atrasadas, concluídas e carga ativa por colaborador). Não há camada analítica persistida.
- **Segurança:** RBAC no cliente (isLeader, canManage, canAssign) e regra de visibilidade de tarefas privadas. Na proposta, autenticação via provedor de identidade e autorização no servidor.
- **Observabilidade:** inexistente na versão atual; o Activity Log cobre apenas auditoria funcional. Proposta descrita na seção 11.

4.2 Componentes da Arquitetura

Componente	Função	Tecnologia	Responsável
main.tsx / index.html	Ponto de entrada: monta o App em StrictMode e carrega fontes e CSS global	React DOM 18, Vite 5	Time Front-end
App	Orquestra a visão ativa (Planner, Kanban, Painel), os filtros por colaborador e projeto e a abertura dos modais	React (useState, useMemo)	Time Front-end
Layout (Sidebar, Topbar)	Navegação, filtros, menu exclusivo do líder e troca simulada de sessão	React, Tailwind, lucide-react	Time Front-end
Calendário (CalendarView, MonthView, WeekView, DayView)	Exibe tarefas por prazo, navega entre períodos, cria tarefa rápida e reagenda por arrastar e soltar	React, date-fns 3 (ptBR), HTML5 Drag and Drop	Time Front-end
KanbanBoard	Quadro com 4 colunas de status e mudança de status por arrastar e soltar	React, HTML5 Drag and Drop	Time Front-end
Módulo Tarefa (TaskCard, TaskModal, NewTaskModal, SubtaskList, CommentsThread, ActivityLog)	Cartão resumido, criação, edição, checklist com progresso, comentários e histórico da tarefa	React, Tailwind	Time Front-end
TeamDashboard	Indicadores da equipe e carga de trabalho ativa, atrasada e concluída por colaborador (somente líder)	React (useMemo)	Time Front-end
UI Primitives	Componentes visuais reutilizáveis: Avatar,	React, Tailwind, clsx	Time Front-end

Componente	Função	Tecnologia	Responsável
	AvatarStack, Badge, Button, Modal e ProgressBar		
useAppStore	Fonte única de verdade: sessão, visibilidade, ações de CRUD, cálculo de progresso e registro de atividades	Zustand 4.5 + middleware persist	Time Front-end
types.ts / lib/utils.ts	Modelo de domínio tipado (espelha o Prisma) e funções auxiliares de datas, iniciais, prioridade, atraso e geração de IDs	TypeScript 5 (strict)	Time Front-end
mockData	Carga inicial: 5 usuários, 3 projetos, 10 tarefas, 13 subtarefas, 4 comentários e 5 atividades	TypeScript	Time Front-end
API Back-end (proposto)	Expor as ações da store como endpoints REST, validar payloads e aplicar RBAC no servidor	Node.js + TypeScript, Zod	A definir (Time Back-end)
Banco de Dados (proposto)	Persistência relacional das 8 entidades do domínio	PostgreSQL + Prisma 5	A definir (Time Back-end / DBA)

5. Modelo C4

5.1 Contexto (C1)

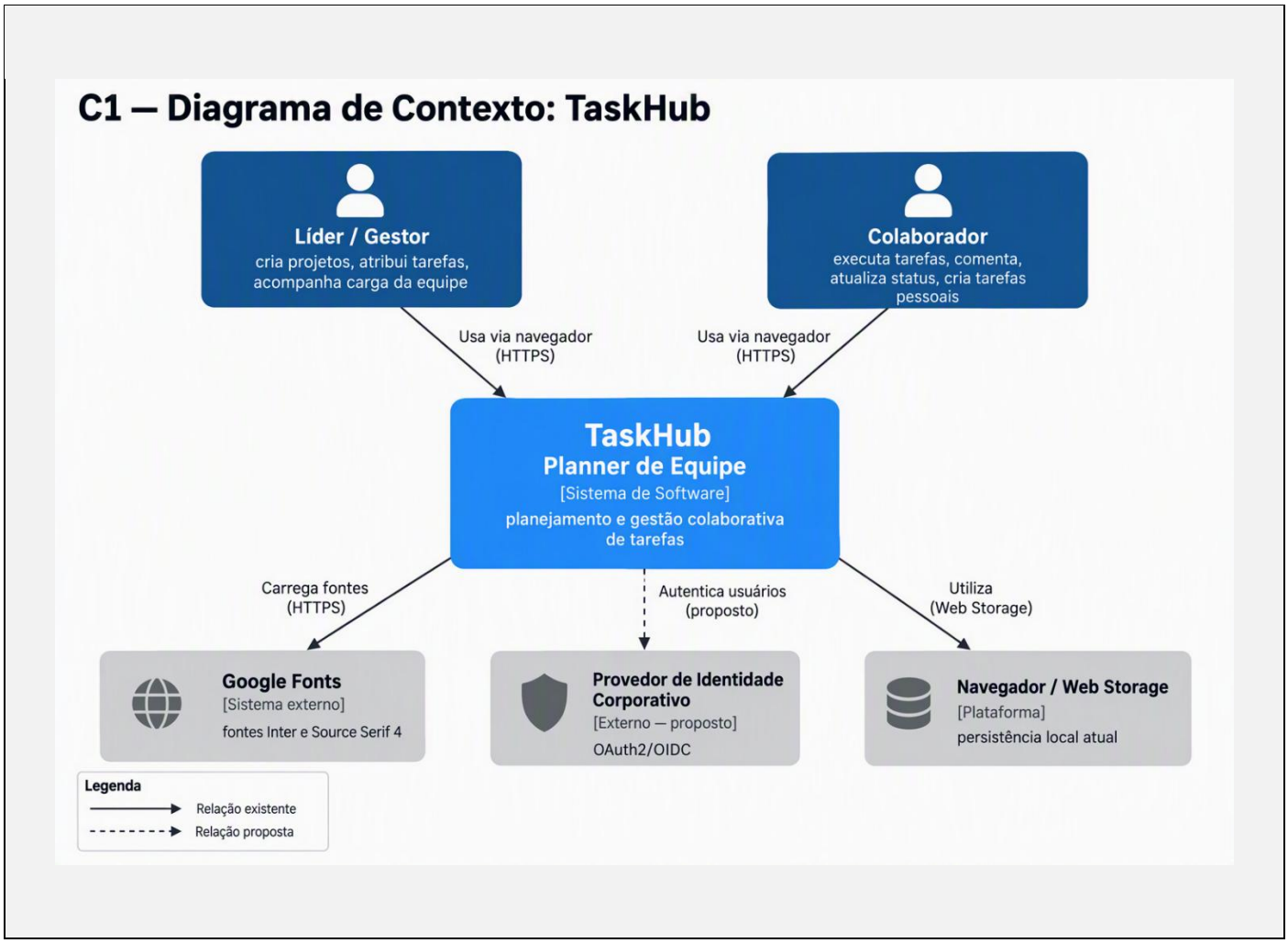
Objetivo

Mostrar o TaskHub como um único sistema e sua relação com as pessoas que o utilizam e com os sistemas externos dos quais depende. Na versão atual, o sistema roda inteiramente no navegador do usuário e só depende do Google Fonts. Na arquitetura alvo, entra também um provedor de identidade corporativo.

Elementos

- **Usuários:** Líder/Gestor (LEADER), que cria e atribui tarefas de equipe, edita qualquer tarefa e acessa o Painel da Equipe; e Colaborador (MEMBER), que executa e move tarefas, comenta, usa o checklist e cria tarefas pessoais.
- **Sistemas externos:** Google Fonts (fontes Inter e Source Serif 4) e o navegador, com a Web Storage API como persistência atual.
- **Integrações:** hoje, nenhuma integração de negócio. Na proposta, provedor de identidade (OAuth2/OIDC) para autenticação.

Diagrama C1



5.2 Contêineres (C2)

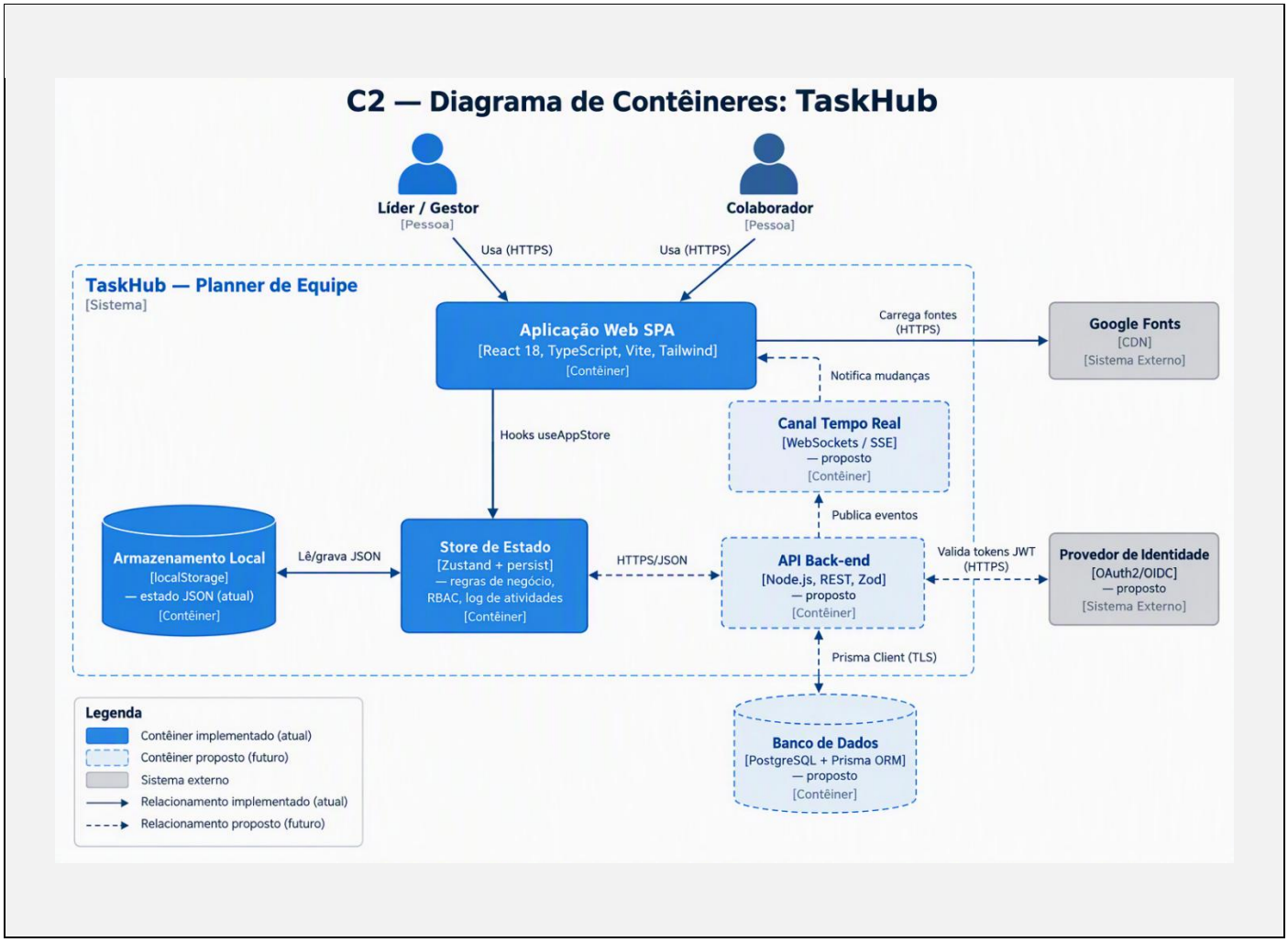
Objetivo

Mostrar os blocos executáveis do sistema, as tecnologias de cada um e como se comunicam. Os contêineres marcados como "proposto" ainda não existem no código; eles representam a evolução indicada no README.

Elementos

- **Front-end:** SPA React 18 + TypeScript, empacotada pelo Vite 5 (build: tsc -b && vite build) e estilizada com Tailwind CSS 3 via PostCSS/Autoprefixer. O alias de importação "@/" aponta para src/.
- **Back-end:** inexistente na v1.0.0. Proposta: API REST em Node.js/TypeScript, com validação Zod e Prisma Client.
- **Banco de Dados:** hoje, localStorage (estado JSON). Proposta: PostgreSQL, com schema e migrations gerenciados pelo Prisma (prisma generate / prisma migrate dev).
- **Integrações:** Google Fonts (atual); provedor de identidade OAuth2/OIDC (proposto).
- **Mensageria:** não existe hoje. Proposta: WebSockets ou Server-Sent Events para propagar comentários e atividades entre usuários conectados.

Diagrama C2



5.3 Componentes (C3)

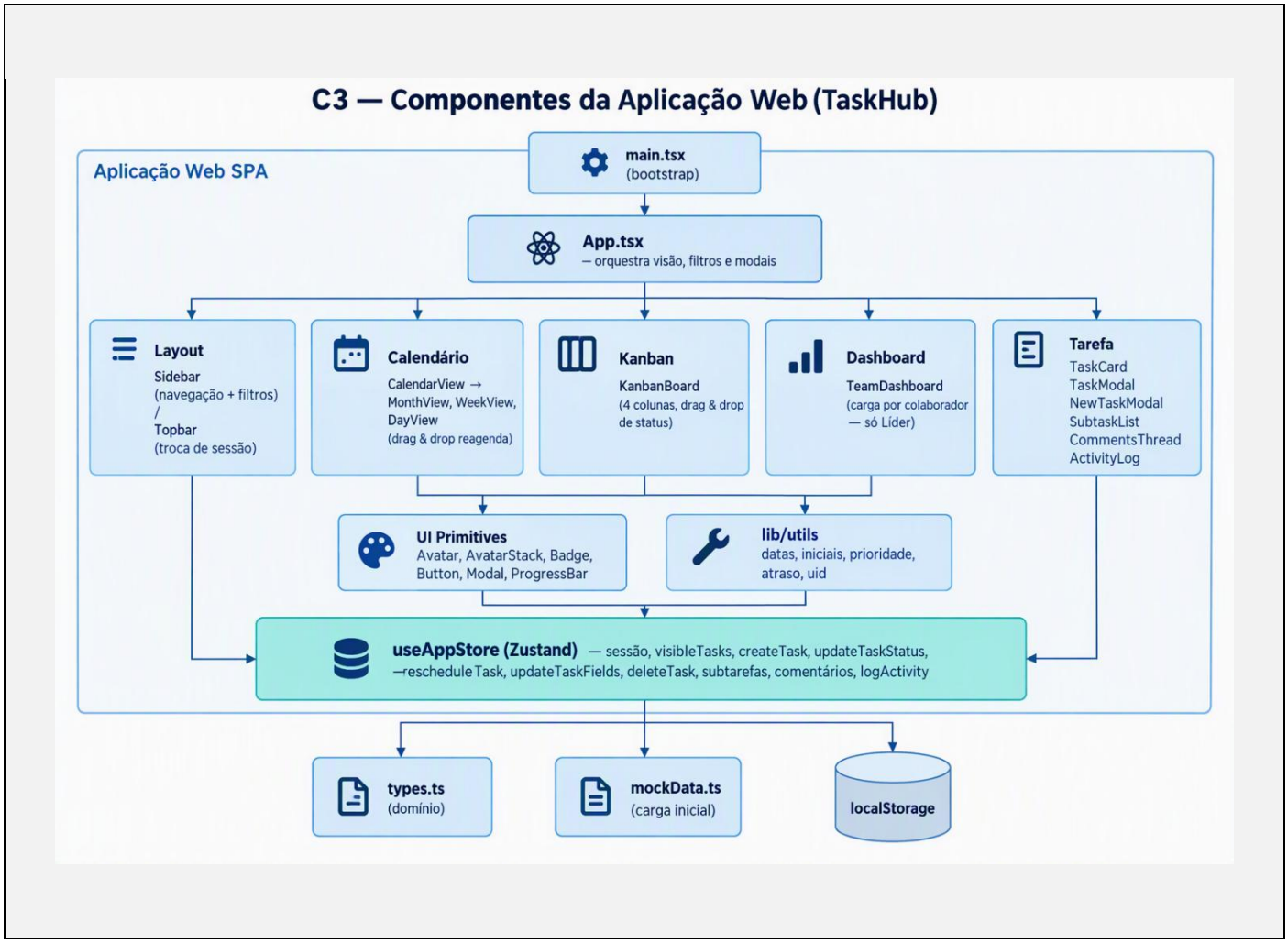
Objetivo

Detalhar a estrutura interna da Aplicação Web. Como não há back-end implementado, os papéis clássicos (controllers, services, repositories) foram mapeados para os elementos equivalentes no front-end.

Componentes

- **Controllers:** App.tsx (estado de navegação e filtros) e os componentes de tela (CalendarView, KanbanBoard, TeamDashboard, TaskModal, NewTaskModal), que tratam eventos do usuário e acionam a store.
- **Services:** ações da useAppStore: switchUser, visibleTasks, createTask, updateTaskStatus, rescheduleTask, updateTaskFields, deleteTask, addSubtask, toggleSubtask, removeSubtask, addComment e taskProgress, além da função interna logActivity.
- **Repositories:** o middleware persist do Zustand (chave "team-planner-storage" no localStorage) e os seletores de leitura subtasksForTask, commentsForTask e activityForTask. Na proposta, serão substituídos por repositórios com Prisma Client.
- **Jobs:** não há rotinas agendadas. O cálculo de atraso (isOverdue) é feito a cada renderização.
- **APIs:** não há APIs expostas na v1.0.0. O contrato proposto está na seção 8.2.

Diagrama C3



6. Arquitetura de Dados

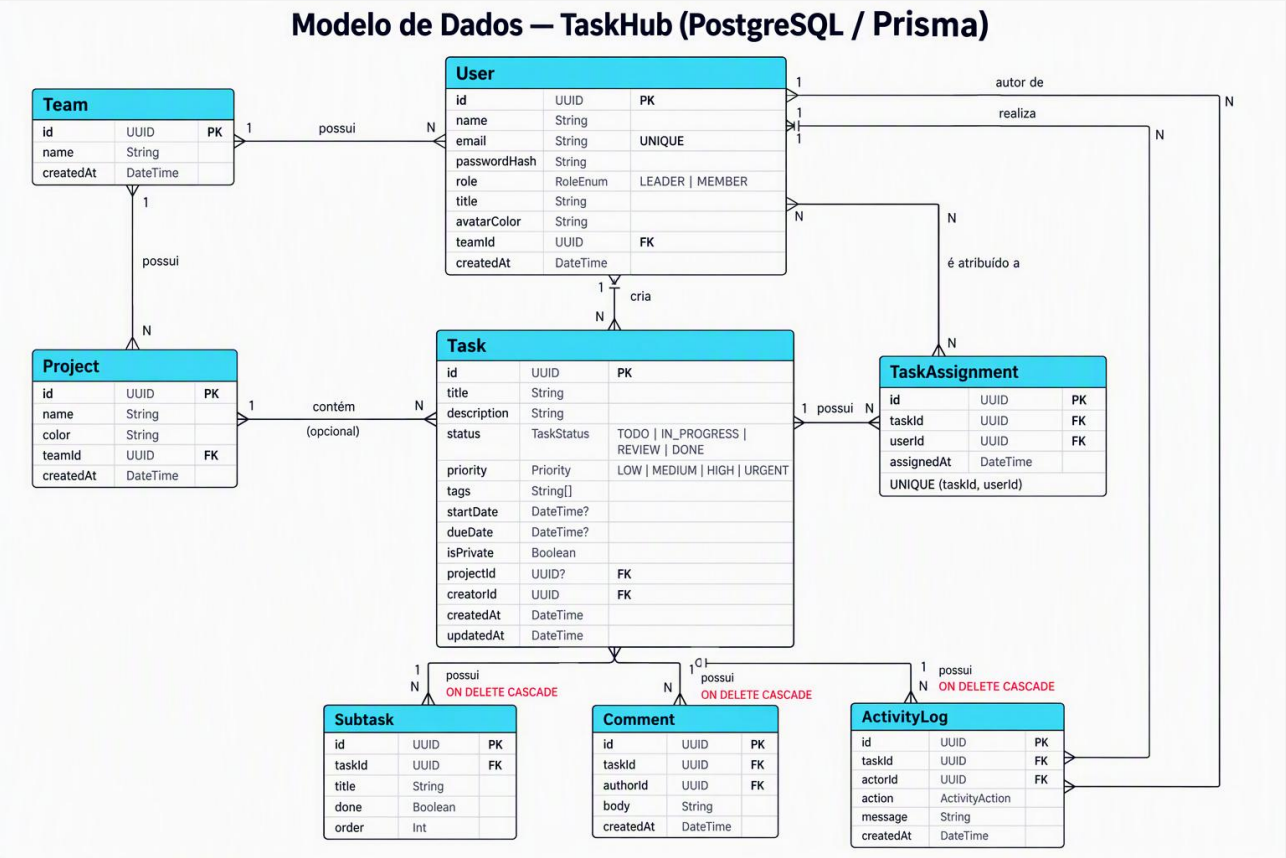
6.1 Fontes de Dados

Fonte (nome do sistema de origem)	Tipo (Data Lake, Banco, API, Arquivo, ERP, IoT, SaaS)	Frequência de atualização	Nome da Tabela/Entidade	Volume estimado	Responsável / Owner dos dados
mockData.ts (atual)	Arquivo (código-fonte)	Carga única na inicialização ou no resetToMock	users, projects, tasks, subtasks, comments, activity	40 registros	Time Front-end
localStorage do navegador (atual)	Arquivo (JSON no Web Storage)	Tempo real (a cada alteração da store)	team-planner-storage (AppState completo)	Limitado a ~5 MB por origem	Usuário (dados locais)
PostgreSQL (proposto)	Banco relacional	Sob demanda (cadastro)	Team, User, Project	A estimar	Líder / Gestor
PostgreSQL (proposto)	Banco relacional	Contínua (uso diário)	Task, TaskAssignment, Subtask, Comment	A estimar	Líder / Gestor e Colaboradores
PostgreSQL (proposto)	Banco relacional	Contínua (gerada por evento)	ActivityLog	A estimar (maior crescimento)	Sistema (auditoria)

6.2 Modelo de Dados

- **Dados Mestres:** Team (equipe), User (nome, e-mail único, hash de senha, papel LEADER/MEMBER, cargo e cor do avatar) e Project (nome e cor).
- **Dados Transacionais:** Task (título, descrição em markdown, status, prioridade, etiquetas, data de início, prazo, privacidade, projeto e criador); TaskAssignment (relação N:N entre tarefa e responsáveis, única por par); Subtask (título, concluída e ordem); Comment (autor, texto e data); e ActivityLog (ator, ação, mensagem e data). Os filhos de Task são removidos em cascata.
- **Dados Analíticos:** não persistidos. O Painel da Equipe calcula em memória total de tarefas de equipe, em andamento, atrasadas e concluídas, além da carga por colaborador. O progresso da tarefa é o percentual de subtarefas concluídas.
- **Enumerações:** Role (LEADER, MEMBER); TaskStatus (TODO, IN_PROGRESS, REVIEW, DONE); TaskPriority (LOW, MEDIUM, HIGH, URGENT); ActivityAction (CREATED, STATUS_CHANGED, DUE_DATE_CHANGED, ASSIGNED, UNASSIGNED, PRIORITY_CHANGED, COMMENTED, SUBTASK_TOGGLED).
- **Índices:** User(teamId), Project(teamId), Task(projectId, dueDate, status), TaskAssignment(userId), Subtask(taskId), Comment(taskId) e ActivityLog(taskId).

Diagrama de Dados


Modelo de Dados — TaskHub (PostgreSQL / Prisma)
Team
id UUID PK
name String
createdAt DateTime
User
id UUID PK
name String
email String UNIQUE
passwordHash String
role RoleEnum LEADER | MEMBER
title String
avatarColor String
teamId UUID FK
createdAt DateTime
Project
id UUID PK
name String
color String
teamId UUID FK
createdAt DateTime
Task
id UUID PK
title String
description String
status TaskStatus TODO | IN_PROGRESS | REVIEW | DONE
priority Priority LOW | MEDIUM | HIGH | URGENT
tags String[]
startDate DateTime?
dueDate DateTime?
isPrivate Boolean
projectId UUID? FK
creatorId UUID FK
createdAt DateTime
updatedAt DateTime
TaskAssignment
id UUID PK
taskId UUID FK
userId UUID FK
assignedAt DateTime
UNIQUE (taskId, userId)
Subtask
id UUID PK
taskId UUID FK
title String
done Boolean
order Int
Comment
id UUID PK
taskId UUID FK
authorId UUID FK
body String
createdAt DateTime
ActivityLog
id UUID PK
taskId UUID FK
actorId UUID FK
action ActivityAction
message String
createdAt DateTime
possui
autor de
realiza
é atribuído a
cria
contém
(opcional)
possui
ON DELETE CASCADE
possui
ON DELETE CASCADE
possui
ON DELETE CASCADE

7. Integrações

7.1 Integrações Internas

Sistema	Tipo	Direção	Protocolo
Componentes React ↔ Store Zustand	Chamada em processo (síncrona)	Bidirecional	Hooks useAppStore (seletores e actions)
Store Zustand ↔ localStorage	Persistência local	Bidirecional	Web Storage API (JSON serializado)
Front-end ↔ API Back-end (proposto)	API síncrona	Bidirecional	HTTPS / REST / JSON + Bearer JWT
API Back-end ↔ PostgreSQL (proposto)	Acesso a dados	Bidirecional	Prisma Client sobre TCP/TLS (DATABASE_URL)
API Back-end → Clientes conectados (proposto)	Eventos assíncronos	Saída	WebSockets ou Server-Sent Events

7.2 Integrações Externas

Sistema	Tipo	Direção	Protocolo
Google Fonts	CDN (SaaS)	Entrada (download de fontes)	HTTPS (fonts.googleapis.com)
Registro npm	Repositório de pacotes (apenas build)	Entrada	HTTPS (registry.npmjs.org)
Provedor de Identidade Corporativo (proposto)	Autenticação (SSO)	Bidirecional	OAuth 2.0 / OpenID Connect sobre HTTPS

7.3 Mensageria

- **Filas:** não utilizadas na versão atual nem necessárias no escopo proposto.
- **Eventos:** hoje, a reatividade é interna (assinaturas do Zustand atualizam os componentes ao alterar o estado). Na proposta, a API publica eventos de domínio (tarefa criada, status alterado, prazo alterado, comentário adicionado, item do checklist alternado) via WebSockets/SSE, e a store mescla o estado recebido.
- **Webhooks:** não aplicável na versão atual. Uso futuro possível para notificar ferramentas externas (por exemplo, Microsoft Teams) sobre tarefas atrasadas.

8. API

8.1 Padrões

- **REST:** padrão adotado (proposto). Recursos no plural, JSON, datas em ISO 8601 (UTC) e códigos HTTP padronizados (200, 201, 204, 400, 401, 403, 404, 409, 422). Payloads validados com Zod, espelhando o NewTaskInput da store.
- **GraphQL:** não adotado. O domínio é pequeno e as consultas são previsíveis.
- **gRPC:** não adotado. Não há comunicação entre serviços internos.

8.2 Catálogo de APIs

Método	Endpoint	Descrição
GET	/api/me	Retorna o usuário autenticado e seu papel (substitui currentUser/isLeader)
GET	/api/users	Lista os membros da equipe do usuário
GET / POST	/api/projects	Lista projetos da equipe / cria projeto (somente LEADER)
GET	/api/tasks?assigneeId=&projectId=&from=&to=	Lista as tarefas visíveis ao usuário (regra de visibleTasks), com filtros por responsável, projeto e período
POST	/api/tasks	Cria tarefa (createTask) e registra a atividade CREATED
GET	/api/tasks/{id}	Detalhe da tarefa com responsáveis e progresso
PATCH	/api/tasks/{id}/status	Altera o status (updateTaskStatus) e registra STATUS_CHANGED
PATCH	/api/tasks/{id}/schedule	Reagenda início e prazo (rescheduleTask) e registra DUE_DATE_CHANGED
PATCH	/api/tasks/{id}	Atualiza título, descrição, prioridade e responsáveis (updateTaskFields)
DELETE	/api/tasks/{id}	Exclui a tarefa e, em cascata, subtarefas, comentários e atividades (LEADER ou criador)
POST	/api/tasks/{id}/subtasks	Adiciona item ao checklist (addSubtask)
PATCH / DELETE	/api/subtasks/{id}	Marca ou desmarca o item (toggleSubtask, registra SUBTASK_TOGGLED) / remove o item (removeSubtask)
GET / POST	/api/tasks/{id}/comments	Lista comentários em ordem cronológica / adiciona comentário (addComment, registra COMMENTED)
GET	/api/tasks/{id}/activity	Histórico da tarefa, do mais recente para o mais antigo (activityForTask)
GET	/api/dashboard/workload	Indicadores da equipe e carga por colaborador (somente LEADER)

8.3 Segurança das APIs

- **OAuth2:** fluxo Authorization Code com PKCE para a SPA (proposto).
- **JWT:** access token de curta duração enviado no cabeçalho Authorization: Bearer, com refresh token e validação de assinatura, expiração, audiência e emissor em cada requisição (proposto).
- **API Key:** não recomendada para acesso de usuários; reservada a eventuais integrações sistema a sistema.
- **OpenID Connect:** identidade do usuário (id_token) obtida do provedor corporativo; papel (LEADER/MEMBER) e equipe resolvidos no banco do sistema (proposto).

8.4 OpenAPI / Swagger

Ainda não existe especificação OpenAPI, pois a API não foi implementada. Recomenda-se gerar o arquivo openapi.yaml a partir dos schemas Zod (por exemplo, com zod-to-openapi), versioná-lo no repositório e publicá-lo via Swagger UI no ambiente de homologação.

9. Arquitetura de IA e MLOps (Se aplicável)

9.1 Modelos Analíticos

Não aplicável. O sistema não utiliza modelos analíticos, de machine learning ou de IA generativa. Os indicadores do Painel da Equipe são calculados por regras determinísticas (contagens e percentuais).

9.2 Fluxo de Treinamento

Não aplicável.

9.3 Fluxo de Inferência

Não aplicável.

9.4 Governança de Modelos

Não aplicável.

9.5 Monitoramento

Não aplicável.

10. Segurança

10.1 Princípios

- **Menor privilégio:** o colaborador só altera o que criou ou o que lhe foi atribuído; atribuição de tarefas de equipe e Painel da Equipe são exclusivos do líder; tarefas privadas são visíveis apenas ao criador, inclusive para o líder.
- **Zero Trust:** toda requisição à API deverá ser autenticada e autorizada no servidor. As verificações atuais na interface (isLeader, canManage, canAssign) são apenas de experiência de uso e não constituem controle de segurança.
- **Defense in Depth:** HTTPS em todo o tráfego, validação de entrada com Zod, consultas parametrizadas via Prisma, escape automático do React contra XSS, Content Security Policy, segredos fora do código e banco sem acesso público.

10.2 Autenticação

Situação atual: não há autenticação. O Topbar permite "simular sessão como" qualquer usuário (switchUser), e o usuário ativo fica salvo no localStorage. O schema prevê o campo passwordHash na entidade User. **Proposta:** usar SSO com o provedor de identidade corporativo (OAuth 2.0 / OpenID Connect, Authorization Code + PKCE), sem armazenar senhas no sistema (passwordHash passa a ser opcional ou é removido). A API valida o JWT em cada requisição e resolve o usuário pelo e-mail ou identificador do provedor. O seletor de troca de sessão deve ser removido em produção.

10.3 Autorização (RBAC/ABAC)

O modelo adotado é RBAC com dois papéis (enum Role: LEADER e MEMBER), complementado por regras de atributo (ABAC) baseadas no criador da tarefa (creatorId) e na privacidade (isPrivate). A matriz abaixo mostra o comportamento implementado no código; a coluna de observações indica os ajustes recomendados ao levar as regras para o servidor.

Ação	LEADER	MEMBER	Observações
Ver tarefas de equipe	Sim	Sim (todas)	O README diz que o colaborador vê "apenas suas tarefas", mas o código mostra todas as tarefas de equipe. É preciso definir a regra oficial.
Ver tarefas privadas	Só as próprias	Só as próprias	Regra visibleTasks: o filtro deve ser aplicado na consulta ao banco.
Criar tarefa de equipe	Sim, para qualquer responsável	Sim, apenas para si	O comentário do schema diz que apenas o LEADER cria tarefas de equipe. Divergência a resolver.
Criar tarefa privada	Sim	Sim	O responsável é forçado a ser o próprio criador.
Editar título, descrição, prioridade e datas; excluir	Sim	Só as que criou	Regra canManage. Porém, o reagendamento por arrastar no calendário não verifica essa permissão.
Alterar status	Sim	Sim (qualquer tarefa visível)	O README prevê que o colaborador mova apenas as próprias tarefas. Restringir a responsáveis e criador.

Ação	LEADER	MEMBER	Observações
Atribuir responsáveis	Sim (tarefas de equipe)	Não	Regra canAssign.
Checklist e comentários	Sim	Sim	Sem restrição no código.
Acessar Painel da Equipe	Sim	Não	O item de menu fica oculto para MEMBER (leaderOnly).

10.4 Gestão de Segredos

Situação atual: o front-end não usa segredos. **Proposta:** a string de conexão DATABASE_URL (já referenciada no schema Prisma), os segredos do cliente OAuth e as chaves de assinatura ficam em um cofre gerenciado (por exemplo, Azure Key Vault), são lidos pela API via identidade gerenciada e nunca são versionados no repositório nem expostos ao bundle do Vite (somente variáveis com prefixo VITE_ chegam ao navegador, e elas não podem conter segredos). Rotação periódica e acesso ao cofre por rede privada.

10.5 Proteção de Dados

Dados pessoais tratados: nome, e-mail, cargo e conteúdo de comentários. **Situação atual:** ficam sem criptografia no localStorage, acessíveis a qualquer script da mesma origem e a outras pessoas que usem o mesmo navegador. **Proposta:** TLS 1.2+ em trânsito, criptografia em repouso no PostgreSQL, senhas (se mantidas) apenas como hash forte (bcrypt/argon2), minimização de dados no localStorage (apenas cache não sensível) e política de retenção para comentários e histórico.

10.6 Auditoria

A entidade ActivityLog registra quem fez o quê e quando em cada tarefa: CREATED, STATUS_CHANGED, DUE_DATE_CHANGED, ASSIGNED, PRIORITY_CHANGED, COMMENTED e SUBTASK_TOGGLED. Lacunas identificadas no código: a exclusão de tarefa não é registrada e apaga o próprio histórico; a inclusão e a remoção de subtarefas não são registradas; o tipo UNASSIGNED existe mas não é usado; e a mensagem de prioridade mostra o código interno (ex.: "HIGH") em vez do rótulo. Recomenda-se gravar a auditoria no servidor, de forma imutável, e manter um log de exclusões.

10.7 Compliance

- **LGPD:** base legal de execução de contrato/legítimo interesse para dados de colaboradores; atendimento aos direitos do titular (acesso, correção e eliminação); registro das operações de tratamento; e cuidado com dados pessoais inseridos em textos livres (comentários e descrições).
- **ISO 27001:** controle de acesso, gestão de segredos, registro de eventos e gestão de vulnerabilidades em dependências (npm audit; o lock atual contém pacotes marcados como descontinuados, como eslint 8, glob 7 e rimraf 3).
- **NIST:** alinhamento ao NIST Cybersecurity Framework (Identificar, Proteger, Detectar, Responder, Recuperar) e às diretrizes do NIST SP 800-63 para autenticação.

11. Observabilidade

11.1 Monitoramento

Situação atual: não há monitoramento. **Proposta:** health check da API (/health) e do banco, métricas de disponibilidade, latência p95, taxa de erro e uso de conexões do PostgreSQL, além de monitoramento de experiência no navegador (Core Web Vitals e erros JavaScript), por exemplo com Azure Monitor / Application Insights.

11.2 Logging

Logs estruturados em JSON na API, com níveis (info, warn, error), identificador de correlação por requisição, usuário e rota, sem registrar dados pessoais ou tokens. Erros não tratados do front-end enviados ao serviço de telemetria. O Activity Log continua sendo o histórico funcional visível ao usuário e não substitui o log técnico.

11.3 Tracing

Rastreamento distribuído com OpenTelemetry, propagando o cabeçalho traceparent do navegador para a API e para as consultas do Prisma, para identificar gargalos de ponta a ponta (por exemplo, na listagem de tarefas por período).

11.4 Alertas

Alertas propostos: indisponibilidade da API ou do banco; taxa de erros HTTP 5xx acima de 2% em 5 minutos; latência p95 acima de 1 s; falhas de autenticação em volume anormal; e uso de armazenamento ou conexões do banco acima de 80%. Notificação ao time responsável por e-mail ou Microsoft Teams.

11.5 Dashboards Operacionais

Painel técnico com disponibilidade, requisições por minuto, latência, erros por endpoint e saúde do banco. Painel de uso com usuários ativos, tarefas criadas por dia e eventos por tipo de atividade. O Painel da Equipe do próprio sistema cobre os indicadores de negócio (tarefas atrasadas, em andamento e carga por colaborador).

12. ADRs – Architecture Decision Records

ADR-001

Decisão

Usar uma store global com Zustand (useAppStore), com o middleware persist, como fonte única de verdade e fronteira entre a interface e os dados. Todas as regras de negócio e o registro de atividades ficam nas actions da store.

Contexto

A primeira versão precisava ser totalmente funcional sem back-end, para validar a experiência com os dois perfis. Ao mesmo tempo, devia permitir a troca futura por uma API real sem reescrever os componentes.

Alternativas Avaliadas

- Redux Toolkit: robusto, porém com mais código repetitivo para um domínio pequeno.
- React Context + useReducer: nativo, porém causa re-renderizações amplas e não tem persistência pronta.
- TanStack Query com back-end desde o início: adequado à versão final, mas exigiria API antes de validar a interface.

Justificativa

O Zustand é leve, tipado, tem seletores granulares (menos re-renderizações) e persistência nativa em localStorage. Os componentes dependem apenas de hooks (useAppStore((s) => ...)), então cada action pode passar a chamar a API de forma otimista sem alterar a interface.

Consequências

- **Positivas:** protótipo funcional imediato, baixo acoplamento entre interface e dados e caminho claro de evolução.
- **Negativas:** dados presos ao navegador, sem colaboração real; regras de permissão apenas no cliente; e seletores como currentUser() e visibleTasks() retornam objetos novos a cada chamada, o que pode gerar re-renderizações extras. Esses pontos precisam ser tratados na migração para a API.

ADR-002

Decisão

Adotar PostgreSQL como banco de produção, acessado pelo Prisma ORM, com o modelo definido em prisma/schema.prisma. Os tipos do front-end (types.ts) devem espelhar esse schema.

Contexto

O domínio é fortemente relacional (equipes, usuários, projetos, tarefas com vários responsáveis, subtarefas, comentários e histórico), precisa de integridade referencial e de exclusão em cascata, e usa listas de etiquetas por tarefa.

Alternativas Avaliadas

- SQLite: aceito apenas para protótipo local; não suporta enums nem arrays nativos (exige tabela Tag N:N e validação via Zod).
- Banco NoSQL de documentos: flexível, porém mais fraco em integridade referencial e consultas relacionais.
- SQL puro ou query builder: mais controle, porém sem migrations tipadas nem geração de cliente.

Justificativa

O PostgreSQL oferece enums, arrays (tags), integridade referencial e índices adequados às consultas por equipe, prazo e status. O Prisma gera um cliente TypeScript tipado, coerente com o restante da stack, e gerencia migrations versionadas (prisma migrate).

Consequências

Necessidade de um back-end Node.js para hospedar o Prisma Client (o pacote está nas dependências do front-end e deve ser movido para o projeto da API). Há divergências de modelo a alinhar: Task.description é obrigatória no schema; os tipos do front-end não possuem Team/teamId; e os responsáveis são um array assigneeIds no cliente, mas uma tabela TaskAssignment no banco.